Formulários: construir para entender

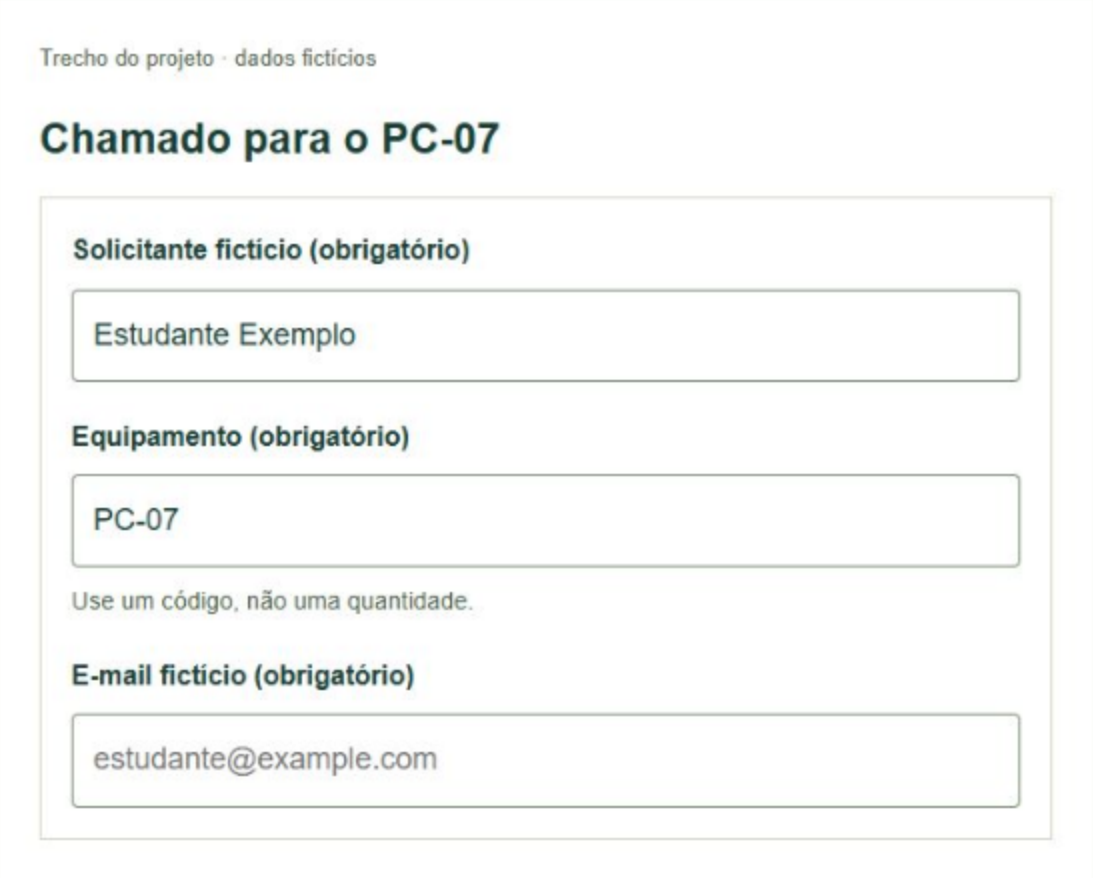
Do pedido de ajuda ao primeiro formulário.
HTML hoje; CSS e JavaScript nos próximos encontros.

Observe → construa → salve → teste → explique

Uma pessoa precisa pedir ajuda

O computador PC-07 está lento. Quais informações ajudam o suporte?

Observe o formulário e relacione cada campo a uma necessidade.



Qual controle combina com o dado?

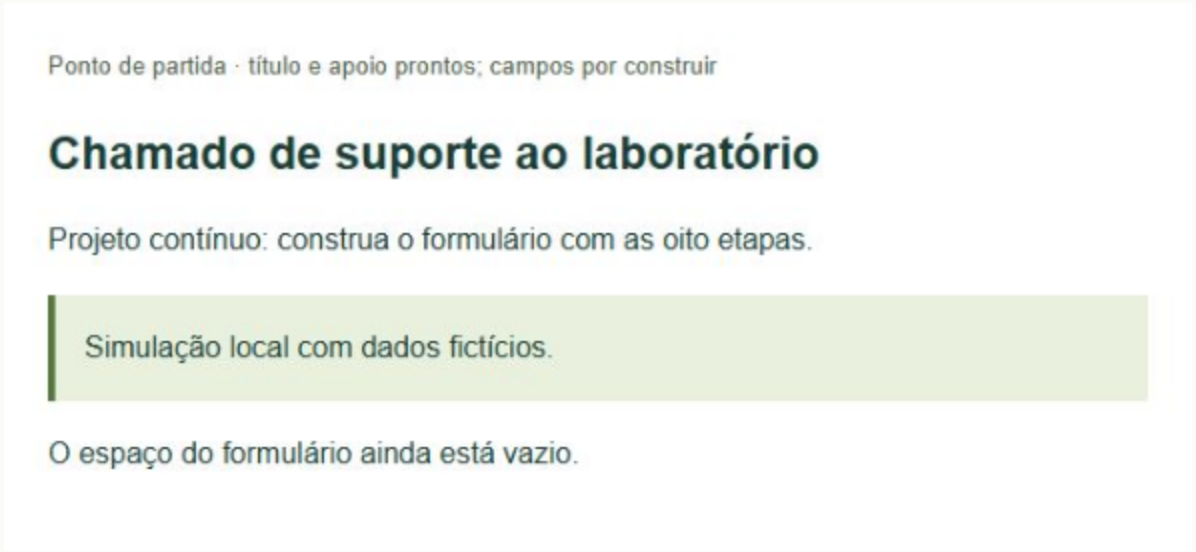
Texto curto	Texto longo	Uma escolha	Várias escolhas
input	textarea	select / radio	checkbox

Texto curto → input. Texto longo → textarea.
Uma opção → select ou radio. Várias opções → checkbox.

Agora: abrir a base no VS Code

Baixe e extraia o projeto da aula 02. Abra a pasta e use Live Server em index.html.

A página começa sem campos. Hoje vamos editar somente index.html.



Tag cria; atributo configura

```
<label for="contato">E-mail fictício</label>
<input id="contato" name="contato"
  type="email" required>
```

label é a tag. for é um atributo. Seu valor precisa coincidir com o id do input.
name identifica o dado; type escolhe o comportamento.

1 · Criar o formulário

index.html: substitua o comentário INÍCIO DO FORMULÁRIO dentro de main.

```
<form id="formulario">
  <div class="campos">
    <!-- CAMPOS: inserir aqui -->
  </div>
  <!-- GRUPOS: inserir aqui -->
  <!-- BOTÃO: inserir aqui -->
</form>
```



Trecho para o quadro · bloco completo no portal

Confira que existe um único form, com abertura e fechamento.

2 · Identificar quem pede ajuda

index.html: antes do comentário CAMPOS, dentro de div.campos. Mantenha o comentário para inserir os próximos campos.

```
<label for="solicitante">Solicitante fictício</label>
<input id="solicitante" name="solicitante"
  type="text" required
  minlength="2" maxlength="60">
```



Trecho para o quadro · bloco completo no portal

Clique no rótulo: o cursor deve ir para a caixa.

3 · Escolher o tipo adequado

index.html: depois do solicitante, antes do comentário CAMPOS.

```
<label for="equipamento">Equipamento</label>
<input id="equipamento" name="equipamento"
  type="text" required>

<label for="contato">E-mail fictício</label>
<input id="contato" name="contato"
  type="email" required>
```



Trecho para o quadro · bloco completo no portal

Digite PC-07 e um e-mail fictício. O teste de formato será feito com o botão na etapa 8.

4 · Oferecer opções com select

index.html: após contato, antes do comentário CAMPOS.

```
<label for="categoria">Categoria
(obrigatório)</label>
<select id="categoria" name="categoria" required>
  <option value="">Escolha uma opção</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
  <option value="rede">Rede</option>
  <option value="outro">Outro</option>
</select>
```



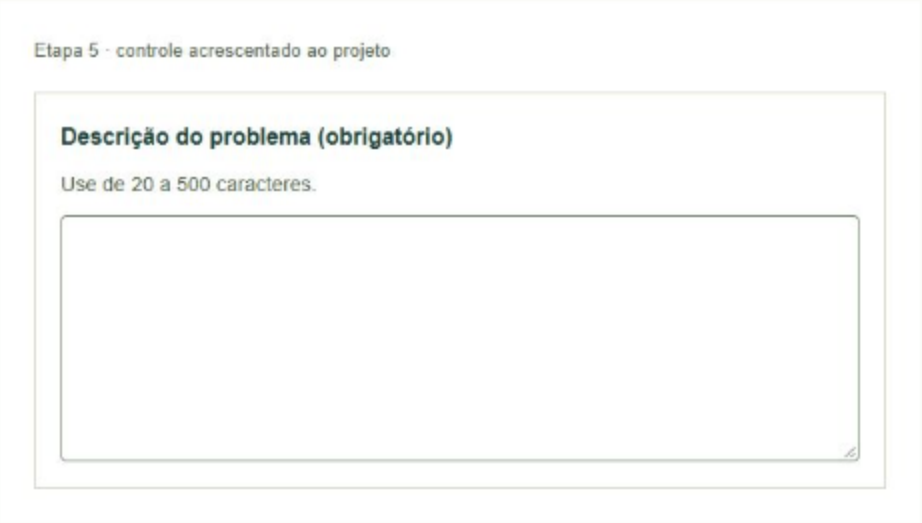
Trecho para o quadro · bloco completo no portal

Abra a lista e escolha Rede. Em 23/10, Outro é apenas uma opção; não abre campo extra.

5 · Dar espaço para a descrição

index.html: depois da categoria, antes do comentário CAMPOS.

```
<label for="descricao">Descrição do problema</label>
<p id="ajuda-descricao">20 a 500 caracteres.</p>
<textarea id="descricao" name="descricao"
  rows="5" required
  minlength="20" maxlength="500"
  aria-describedby="ajuda-descricao"></textarea>
```



Trecho para o quadro · bloco completo no portal

Escreva uma frase que descreva um defeito fictício. O limite será testado após a etapa 8.

6 · Marcar uma única prioridade

index.html: depois do fechamento de div.campos e antes do comentário GRUPOS.

```
<fieldset>
  <legend>Prioridade (obrigatória)</legend>
  <label><input type="radio" name="prioridade"
    value="normal" required> Normal</label>
  <label><input type="radio" name="prioridade"
    value="alta"> Alta</label>
</fieldset>
```



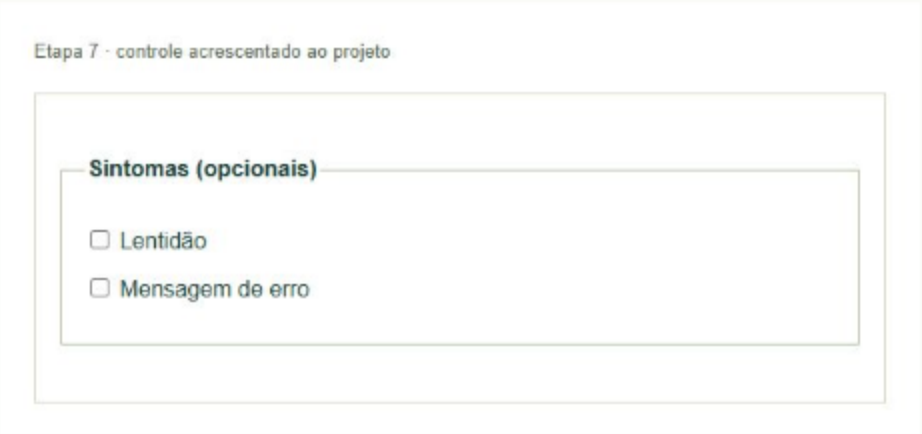
Trecho para o quadro · bloco completo no portal

Marque Normal e depois Alta: a primeira deve ser desmarcada.

7 · Aceitar mais de um sintoma

index.html: após o fieldset de prioridade e antes do comentário GRUPOS.

```
<fieldset>
  <legend>Sintomas (opcionais)</legend>
  <label><input type="checkbox" name="sintomas"
    value="Lentidão"> Lentidão</label>
  <label><input type="checkbox" name="sintomas"
    value="Mensagem de erro">
    Mensagem de erro</label>
</fieldset>
```



Trecho para o quadro · bloco completo no portal

Marque as duas; depois desmarque uma. Nenhum sintoma é obrigatório.

8 · Testar o formulário

index.html: substitua o comentário BOTÃO, ainda dentro de form, depois dos grupos.

```
<div class="acoes">
  <button id="simular" type="submit" disabled>
    Testar campos
  </button>
</div>
```



Trecho para o quadro · bloco completo no portal

Teste vazio, e-mail inválido e dados válidos. O resultado válido é uma mensagem local, sem resumo ainda.

Conferir a estrutura construída

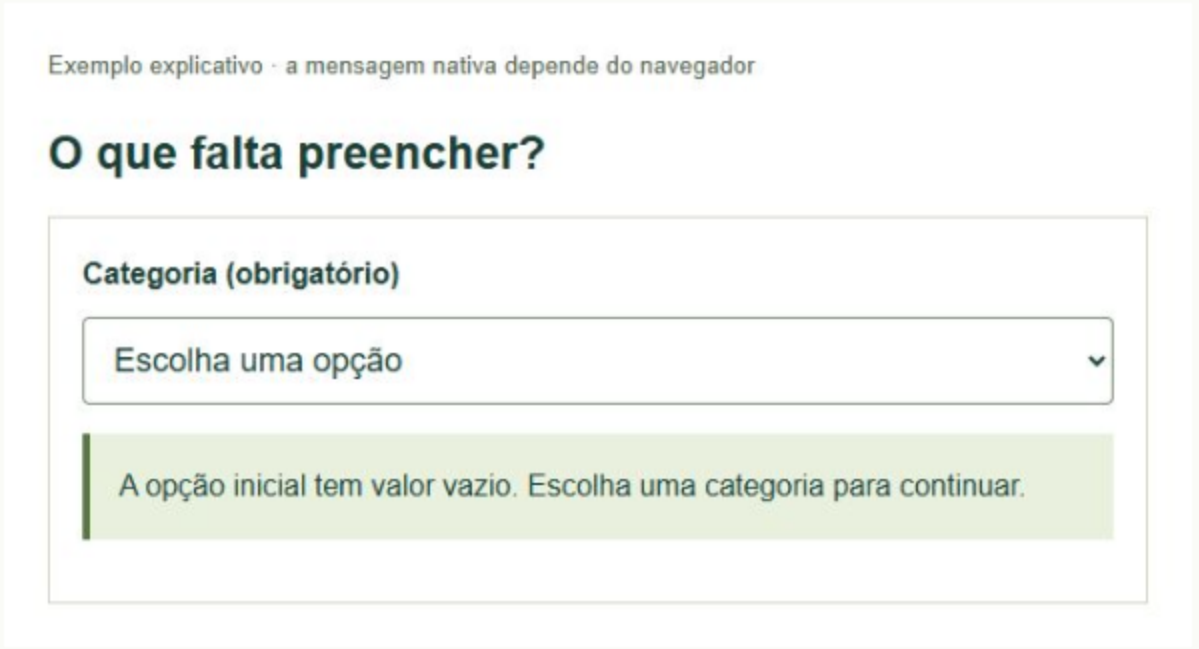
```
<main>
  <form id="formulario">
    <div class="campos">
      <!-- campos de texto e lista -->
    </div>
    <!-- prioridade e sintomas -->
    <!-- botão -->
  </form>
  <p id="status" role="status"></p>
</main>
```

Os campos ficam dentro de div.campos; os dois fieldsets vêm depois dessa div.
Tudo, inclusive o botão, fica dentro de um único form.

Por que o navegador bloqueou?

required exige preenchimento;
type="email" confere o formato.

A descrição exige 20 a 500 caracteres.
Essas regras não provam que os dados são verdadeiros.



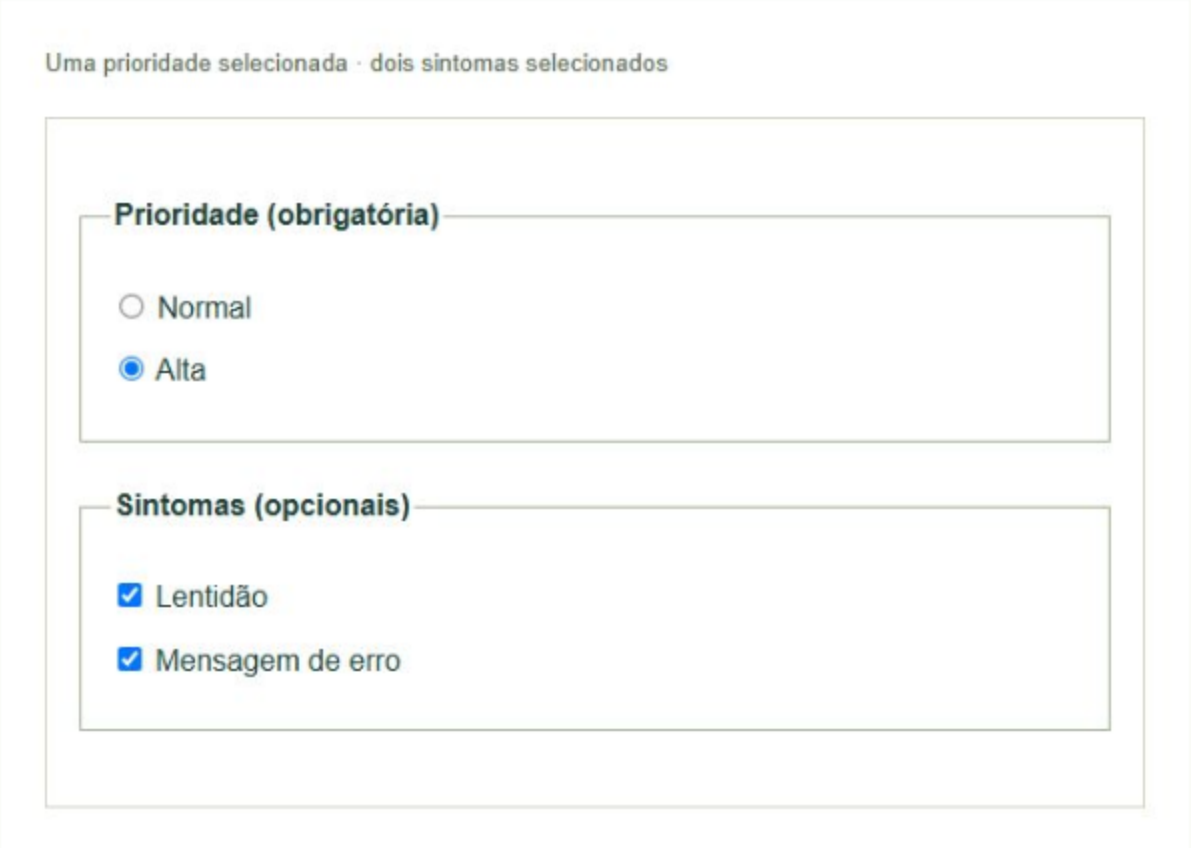
Conferência dos testes

Caso	Resultado esperado
Campos obrigatórios vazios	O navegador impede a simulação.
Contato sem formato de e-mail	O campo de contato fica inválido.
Descrição digitada com menos de 20 caracteres	O navegador pede completar o texto.

Radio ou checkbox?

Prioridade: só uma pode ficar marcada.

Sintomas: duas, uma ou nenhuma podem ficar marcadas.



O nome do dado

for="contato"

id="contato"

name="contato"

chave do dado

id liga o rótulo ao campo. name identifica o dado que será coletado.
Nas próximas aulas, FormData vai ler esses nomes. Hoje confira o HTML.

Agora: conferir sem copiar o modelo

Caso	Resultado esperado
Clique no rótulo do equipamento	O foco vai para o campo correspondente.
Marcar duas prioridades em sequência	Só a última permanece marcada.
Marcar os dois sintomas	As duas opções permanecem marcadas.
Preenchimento válido	Mensagem local; nenhuma navegação de envio.

Registro e entrega

No portal, abra Registrar testes na aula 02 e baixe o TXT identificado.

Entregue o projeto em ZIP com os três arquivos e o TXT durante a aula. Não há envio automático.

Uma explicação para fechar

> Qual é o papel de name? Qual teste mostrou que seu formulário estava funcionando?

> Mostre no seu HTML uma tag, um atributo e um teste que você realizou.

O mesmo projeto continua

23/10 · HTML

Campos e regras

→

30/10 · CSS

Layout e foco

→

06/11 · JavaScript

Coleta e resumo

Guarde sua pasta: em 30/10 vamos melhorar o layout com CSS.
Não exigimos confirmação de e-mail, resumo ou campo condicional hoje.

Um formulário legível em qualquer tela

CSS, responsividade, instruções e navegação pelo teclado
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

CSS, responsividade, instruções e navegação pelo teclado

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Organizar em uma coluna

Na janela estreita, os campos cabem sem rolagem horizontal.

→

2 · Acrescentar duas colunas

Na janela larga, duas colunas; a descrição ocupa a linha inteira.

→

3 · Destacar o foco

O campo em foco tem contorno visível e segue a ordem do HTML.

Siga esta ordem; reteste a cada alteração.

Exemplo: Layout começa pela tela estreita

style.css: confira as regras de .campos e .campo.

```
* { box-sizing: border-box; }
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Duas colunas quando há espaço

style.css: no final, acrescente a media query de 48rem.

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Foco visível e ordem de leitura

style.css: acrescente :focus-visible antes da media query.

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Teste da experiência

> Redimensione para 360, 768 e 1280 pixels. Há corte ou rolagem horizontal?

> Use Tab, Shift+Tab e 200% de zoom. Você enxerga e alcança cada controle?

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Tela de aproximadamente 360px	Uma coluna; controles sem corte lateral.
Tela larga a partir de 48rem	Duas colunas; descrição ocupa a linha.
Tab e Shift+Tab	Foco visível, sequência compreensível.
Ampliação de 200%	Conteúdo legível e controles alcançáveis.
Texto longo na instrução	Quebra de linha sem sobrepor outro campo.

Conferência dos testes (continuação)

Caso	Resultado esperado
Preenchimento inválido	Regras HTML da aula anterior continuam ativas.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual decisão ajudou a tela estreita? Como você conferiu o foco?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Os dados aparecem na página

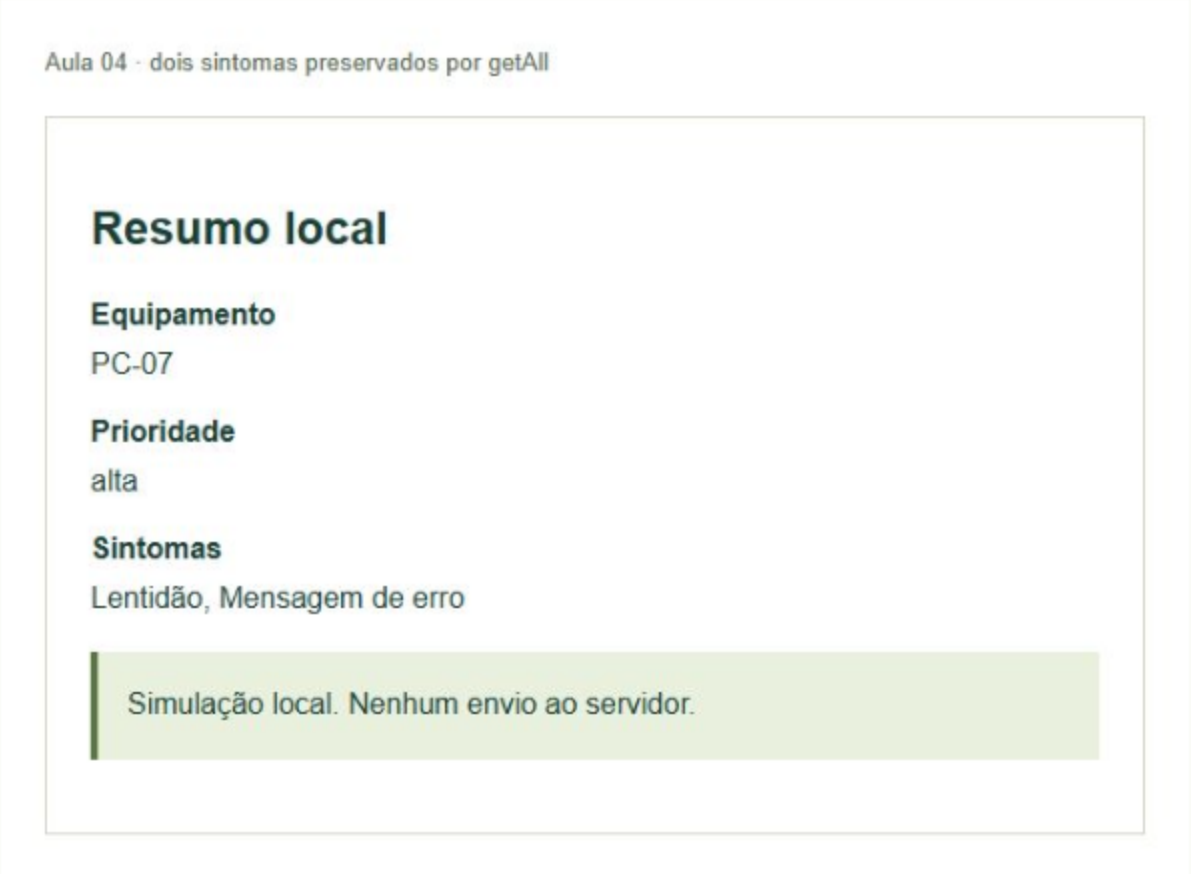
Evento submit, FormData e resumo local com JavaScript
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

Evento submit, FormData e resumo local com JavaScript

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Preparar o resultado

A área do resumo começa oculta; Novo preenchimento permanece acessível.

→

2 · Coletar no submit

Ao marcar dois sintomas, ambos permanecem na lista coletada.

→

3 · Mostrar e reiniciar

Um envio válido mostra resumo local. Novo preenchimento limpa e foca o solicitante.

Siga esta ordem; reteste a cada alteração.

Exemplo: O evento pertence ao formulário

script.js: no único listener de submit, após event.preventDefault().

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  mostrarResumo(dados);
});
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Coleta por name

script.js: no único listener de submit, após event.preventDefault().

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Resumo como conteúdo, sem interpretar HTML

script.js: antes do listener de submit, após selecionar #dados-resumo.

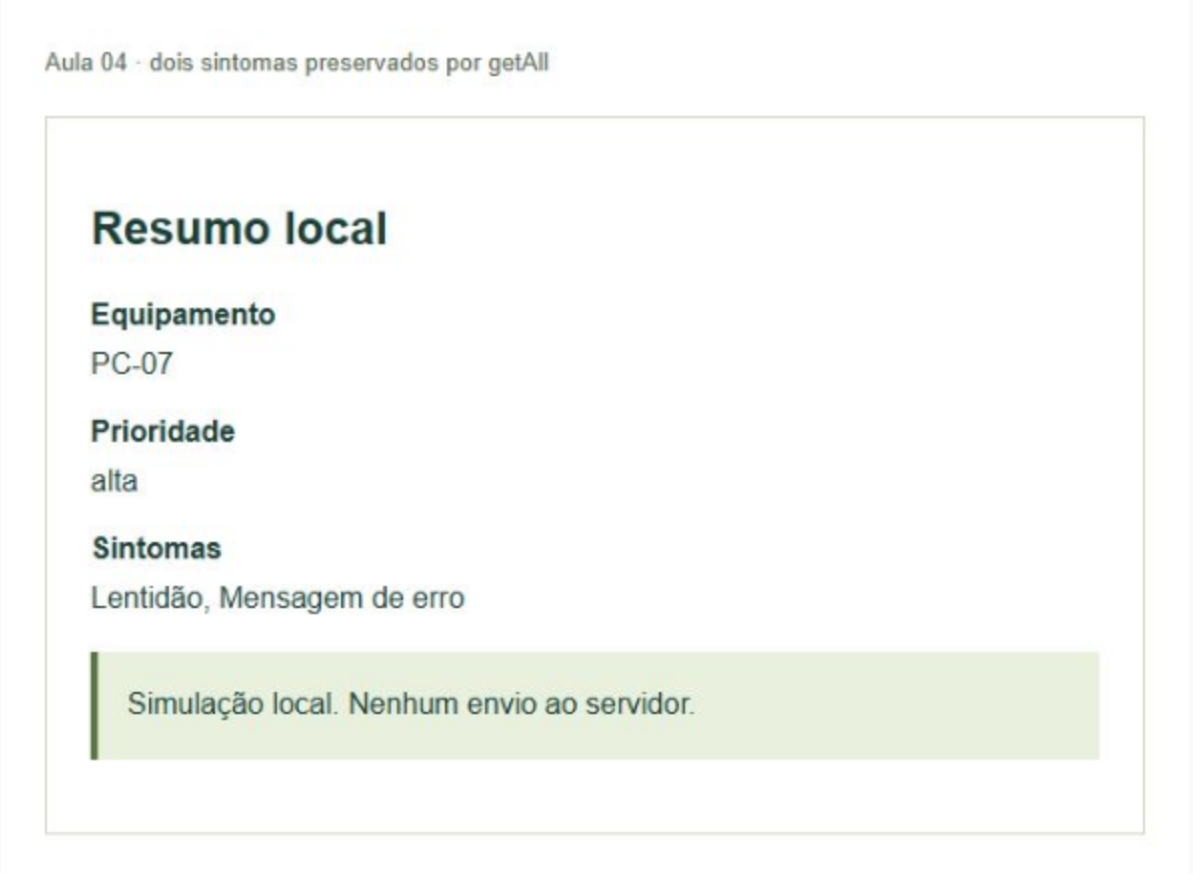
```
function adicionarLinha(titulo, valor) {
  const dt = document.createElement('dt');
  const dd = document.createElement('dd');
  dt.textContent = titulo;
  dd.textContent = String(valor);
  resumo.append(dt, dd);
}
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Estado do formulário depois da tentativa

Tentativa inválida: mantenha os valores. Novo envio válido: substitua o resumo anterior.

Novo preenchimento limpa os campos e devolve o foco. Recarregar a página perde os dados.



Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Campo obrigatório vazio	O resumo não aparece.
Preenchimento válido	Resumo local com os valores informados.
Dois sintomas marcados	Ambos aparecem no resumo.
Nenhum sintoma marcado	Mensagem de nenhuma opção selecionada.
Texto com exemplo na descrição	As marcas aparecem como texto, sem formatação HTML.

Conferência dos testes (continuação)

Caso	Resultado esperado
Duas simulações válidas	O resumo mais recente substitui o anterior.
Novo preenchimento	Campos reiniciados e foco no primeiro campo.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Como o name chega ao FormData? Por que usar textContent no resumo?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Mensagens que ajudam a corrigir

Regras complementares, validade personalizada e casos de teste
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

Regras complementares, validade personalizada e casos de teste

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Adicionar confirmação

O novo campo pede repetição do contato.

→

2 · Comparar e limpar mensagens

Contatos diferentes pedem correção. Ao igualar, o erro desaparece.

→

3 · Conferir antes do resumo

Espaços não completam a descrição. Uma tentativa inválida mantém os valores.

Siga esta ordem; reteste a cada alteração.

Exemplo: Comparação de dois campos

script.js: após selecionar #contato e #confirmar, antes do listener de submit.

```
function conferirContato() {
  const diferente = confirmar.value !== contato.value;
  confirmar.setCustomValidity(
    confirmar.value && diferente ? 'Repita o mesmo e-mail.' : ''
  );
}
contato.addEventListener('input', conferirContato);
confirmar.addEventListener('input', conferirContato);
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Descrição e limites do teste

script.js: após selecionar #descricao, antes do listener de submit.

```
function conferirDescricao() {
  const curta = descricao.value.length > 0
    && descricao.value.trim().length < 20;
  descricao.setCustomValidity(
    curta ? 'Descreva o problema com pelo menos 20 caracteres.' : ''
  );
}
descricao.addEventListener('input', conferirDescricao);
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Nova tentativa sem mensagens antigas

script.js: substitua o listener de submit; mantenha apenas um.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  conferirContato();
  conferirDescricao();
  if (!form.reportValidity()) return;
  mostrarResumo(new FormData(form));
});
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
E-mails diferentes	Mensagem de repetir o mesmo contato.
Corrigir o segundo e-mail	A mensagem antiga é limpa.
Mudar o primeiro depois da confirmação	A divergência volta a ser identificada.
Descrição formada por espaços	A regra complementar bloqueia a simulação.
Descrição digitada com 19 caracteres úteis	A simulação fica bloqueada.

Conferência dos testes (continuação)

Caso	Resultado esperado
Descrição digitada com 20 caracteres úteis	A regra de tamanho permite continuar.
Novo preenchimento após erro	Não permanece mensagem personalizada antiga.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Quando sua mensagem é limpa? Que teste demonstrou a correção do erro anterior?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

GET, POST, HTTPS e contrato do projeto integrador

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Separar simulação e servidor

O projeto continua local, sem abrir uma solicitação real.

→

2 · Representar GET e POST

O navegador mostra dois textos para comparação; não executa uma requisição.

→

3 · Conferir o contrato

A representação de POST não duplica confirmar; HTTPS protege o tráfego, não valida os dados.

Siga esta ordem; reteste a cada alteração.

Os limites do protótipo

> O resumo é gerado no navegador. Os dados não chegam a um servidor de chamados.

> Hoje vamos comparar representações e documentar uma rota futura, sem fazer o envio.

Exemplo: GET e os parâmetros da URL

script.js: trecho dentro de representarEnvio(dados). dados já é o parâmetro da função.

```
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

POST, HTTPS e conferência no servidor

> POST coloca os dados no corpo da requisição. O método sozinho não criptografa os dados.

> HTTPS protege o tráfego. O servidor ainda precisa conferir todas as regras.

Exemplo: Contrato do integrador

index.html: antes de form, registre este comentário sobre a aplicação futura.

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.

Conferência dos testes (continuação)

Caso	Resultado esperado
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Um formulário que reage ao contexto

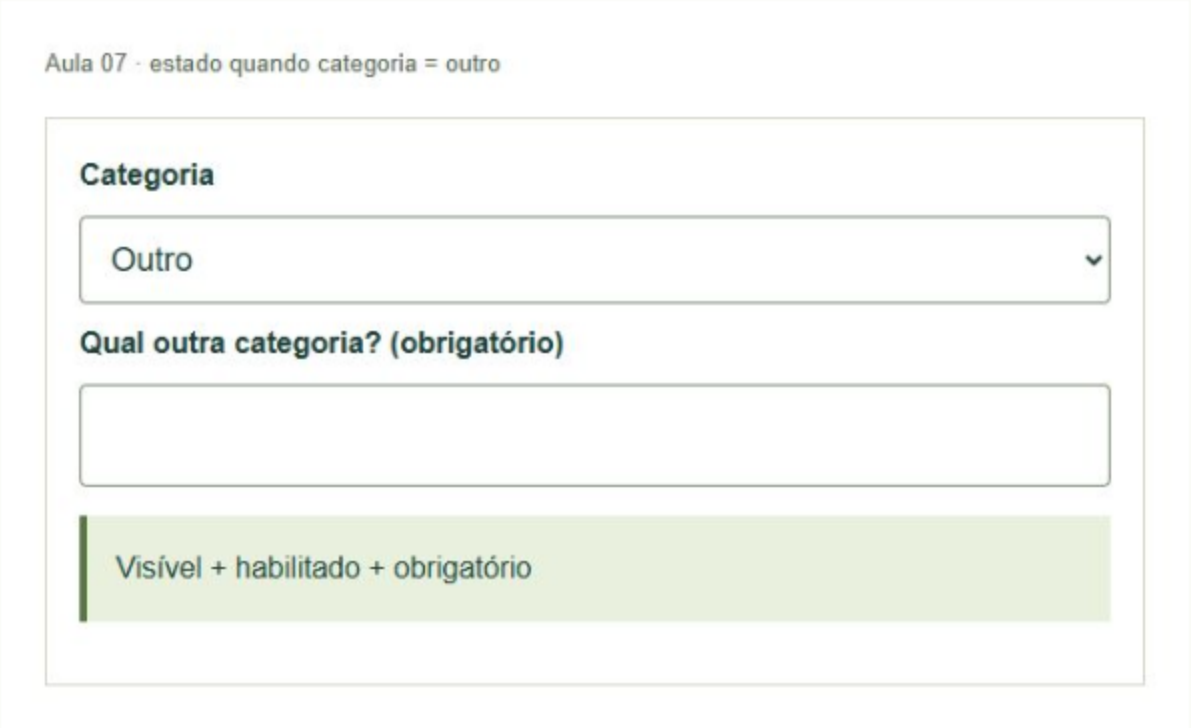
Campos condicionais, estado e organização das funções
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

Campos condicionais, estado e organização das funções

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Adicionar o campo complementar

O campo começa oculto e desativado.

→

2 · Sincronizar o estado

Outro mostra e exige o campo. Hardware esconde e retira o controle da coleta.

→

3 · Repetir o ciclo completo

Nenhum valor antigo reaparece no resumo; reiniciar restaura o estado inicial.

Siga esta ordem; reteste a cada alteração.

Exemplo: A interface responde a uma escolha

script.js: após selecionar categoria, blocoOutro e outro; antes do listener de submit.

```
function atualizarOutro() {
  const ativo = categoria.value === 'outro';
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = '';
}
categoria.addEventListener('change', atualizarOutro);
atualizarOutro();
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Esconder não desativa o controle

script.js: após selecionar categoria, blocoOutro e outro; antes do listener de submit.

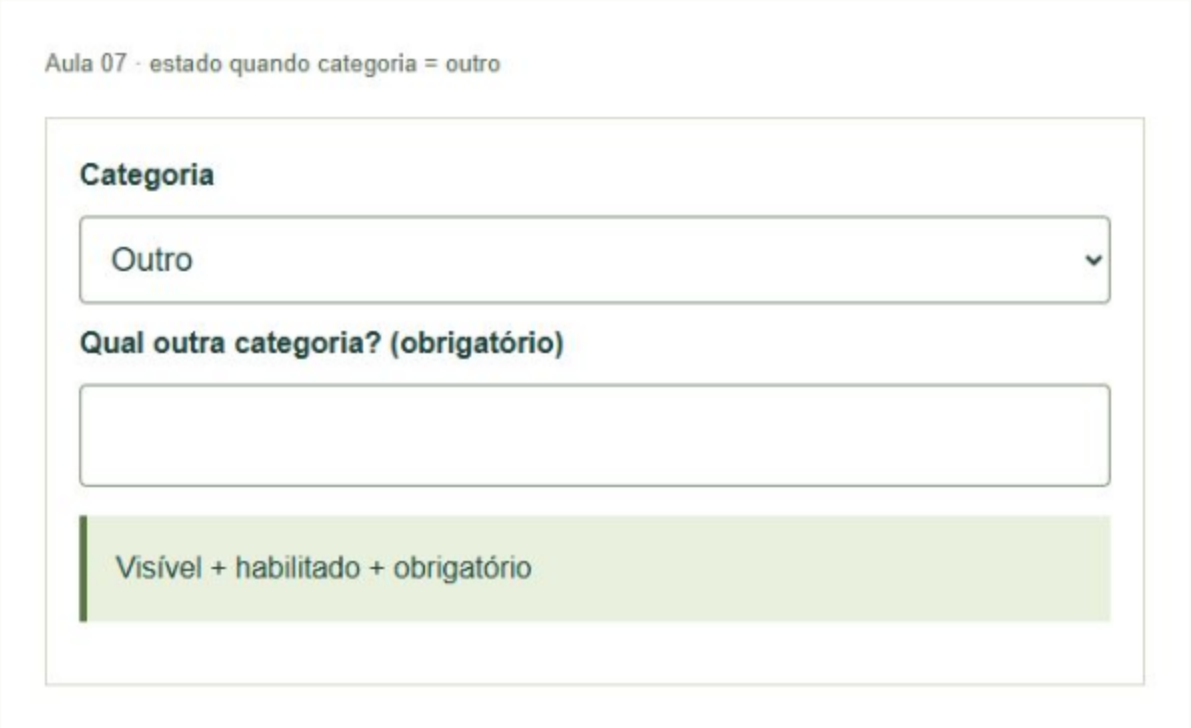
```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Conferência do fluxo inteiro

Teste entrar em Outro, tentar sem preencher, informar um detalhe e gerar o resumo. Depois retorne a Hardware e confira que o detalhe não aparece na coleta.

Use Novo preenchimento para conferir reset e reaplicação do estado. Revise a comparação dos e-mails e as mensagens: acrescentar uma condição não deve quebrar as regras anteriores.



Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Abertura com categoria vazia	Campo adicional oculto e desabilitado.
Escolher Outro sem detalhar	Campo visível e obrigatório.
Outro com detalhe e dados válidos	Resumo inclui a outra categoria.
Trocar de Outro para Hardware	Bloco oculto e valor antigo limpo.
Gerar resumo após sair de Outro	Não inclui outra_categoria.

Conferência dos testes (continuação)

Caso	Resultado esperado
Novo preenchimento	Estado inicial reaplicado.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Que diferença existe entre esconder um campo e desabilitá-lo?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Avaliação prática: inscrição em oficina

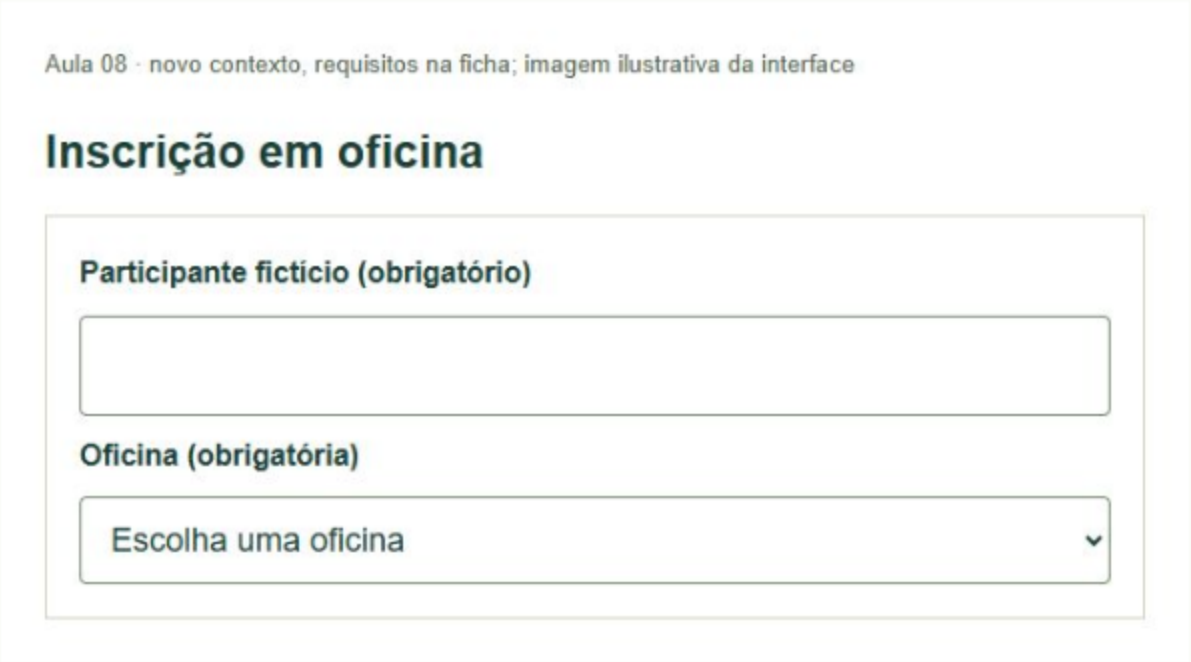
Aplicação individual dos conhecimentos de formulários
Use a nova base da atividade e demonstre o que aprendeu.

Observe → construa → salve → teste → explique

O que vamos construir

Aplicação individual dos conhecimentos de formulários

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Ler antes de implementar

Uma base nova permite demonstrar o que você aprendeu no projeto contínuo.

→

2 · Construir e testar individualmente

Entregue evidências reais; o professor considera código, comportamento e explicação.

Siga esta ordem; reteste a cada alteração.

Critérios propostos para a avaliação

Dimensão	Pontos
HTML e controles	2
CSS e acessibilidade de uso	2
Coleta e resumo local	2
Validação e estado condicional	2
Testes e explicação individual	2

Especificação dos campos

Participante: 2 a 60 caracteres. Contato e confirmação: e-mails obrigatórios e iguais.

Oficina: select obrigatório. Turno: uma escolha em radio. Recursos: escolhas opcionais.

Objetivo: 20 a 500 caracteres úteis. Outra oficina ativa e exige Tema sugerido.

Ao sair de Outra oficina, limpe e desabilite o campo complementar. Confira os nomes na ficha.

Comportamento e entrega

> Mostrar um resumo local com os dados da inscrição, preservando os dois materiais opcionais se marcados. Tratar valores como texto. Não enviar nem gravar em servidor.

> O formulário deve funcionar em tela estreita e pelo teclado. Manter valores após uma tentativa inválida. O reinício precisa limpar as mensagens e o estado condicional. Entregar os três arquivos em ZIP e o registro TXT.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> Use a base específica e cumpra os requisitos da ficha.

Conferência dos testes

Caso	Resultado esperado
Obrigatórios vazios	Bloqueio da tentativa.
E-mails diferentes e depois corrigidos	Erro comunicado e mensagem limpa ao corrigir.
Outra oficina sem tema	Tema obrigatório.
Trocar de Outra oficina para Web	Tema deixa a validação e a coleta.
Caderno e Notebook marcados	Resumo preserva os dois materiais.

Conferência dos testes (continuação)

Caso	Resultado esperado
Texto com sinais de tag no objetivo	Aparece como texto literal.
Tela estreita e teclado	Campos alcançáveis e foco visível.
Novo preenchimento	Regras e estado voltam à condição inicial.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual requisito você demonstrou com maior autonomia? Qual precisa corrigir?

> Guarde o projeto e o registro para a devolutiva e as correções de 18/12.

Corrigir, recuperar e concluir

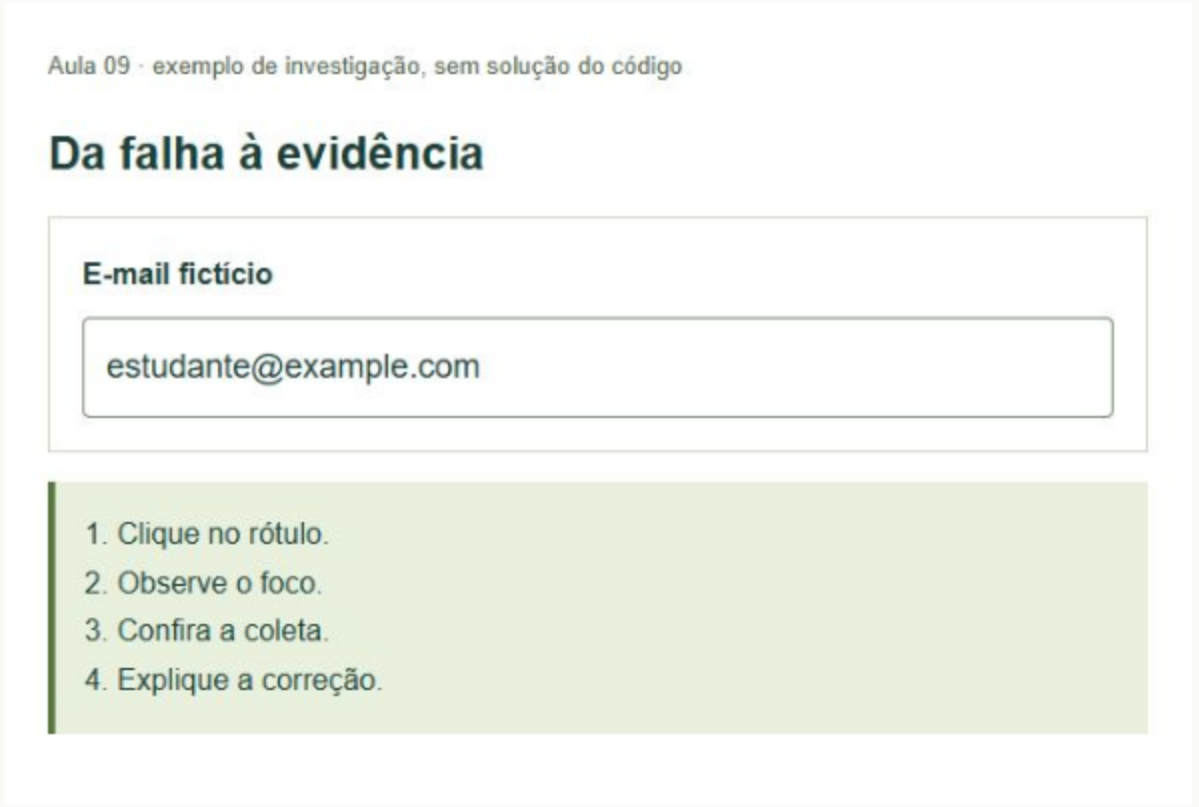
Devolutiva, depuração e nova evidência de aprendizagem
Use a nova base da atividade e demonstre o que aprendeu.

Observe → construa → salve → teste → explique

O que vamos construir

Devolutiva, depuração e nova evidência de aprendizagem

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Investigar antes de alterar

Cada correção tem um teste de regressão; aparência sozinha não comprova funcionamento.

→

2 · Corrigir e demonstrar

Entregue ZIP e TXT com a correção explicada; combine a evidência com a devolutiva individual.

Siga esta ordem; reteste a cada alteração.

Da devolutiva ao plano de correção

> Leia a devolutiva e identifique os requisitos que ainda precisam de evidência.

> Na base de recuperação há defeitos intencionais. Reproduza, corrija e repita o teste.

Exemplo: Investigação de rótulos e coleta

Abra a base de recuperação de 18/12 em uma pasta separada.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Estado e nova tentativa

Ajuste os arquivos da recuperação e reteste os requisitos pendentes da avaliação.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> Use a base específica e cumpra os requisitos da ficha.

Conferência dos testes

Caso	Resultado esperado após a correção
Clicar no rótulo de contato	O foco chega ao contato.
Preencher contato e gerar resumo	O valor informado aparece no resumo.
Escolher Outra oficina	Tema fica habilitado e obrigatório.
Sair de Outra oficina	Tema fica oculto e fora da coleta.
Corrigir um valor inválido	A próxima tentativa pode ocorrer.

Conferência dos testes (continuação)

Caso	Resultado esperado após a correção
Repetir o caso da avaliação	A evidência do critério pendente fica registrada.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual era a causa do erro, como você a corrigiu e qual teste confirmou a mudança?

> Guarde o código e o registro como evidências do que você aprendeu neste período.